

Ideias de Projetos para Banco de Dados

Introdução

Este documento apresenta dez ideias de projetos para a disciplina de Banco de Dados. Os projetos foram pensados para envolver modelagem de banco de dados, diagramas, criação do banco físico e realização de consultas SQL.

Todos os projetos deverão conter muitas tabelas relacionadas e diversas consultas utilizando SELECT. As consultas deverão ser feitas sem o uso de JOIN explícito e sem o uso de SUBSELECT.

1. Requisitos Obrigatórios do Projeto

Cada grupo deverá desenvolver um projeto completo de banco de dados contendo os seguintes itens:

1. Tema do sistema;
2. Descrição do problema;
3. Objetivo do sistema;
4. Regras de negócio;
5. Modelagem conceitual;
6. Modelagem lógica;
7. Diagramas feitos obrigatoriamente no draw.io;
8. Criação do banco de dados físico;
9. Criação das tabelas com chaves primárias e estrangeiras;
10. Inserção de dados fictícios;
11. Consultas SQL sem JOIN e sem SUBSELECT.

A modelagem conceitual deverá representar as principais entidades do sistema, seus atributos e relacionamentos.

Devem ser identificados:

- Entidades;
- Atributos;
- Relacionamentos;
- Cardinalidades;
- Regras principais do domínio escolhido.

1.2. Modelagem Lógica

A modelagem lógica deverá transformar o modelo conceitual em uma estrutura de tabelas. Devem ser definidos:

- Nome das tabelas;
- Atributos de cada tabela;
- Chaves primárias;
- Chaves estrangeiras;
- Relacionamentos;
- Cardinalidades.

1.3. Diagramas no draw.io

Os diagramas deverão ser feitos obrigatoriamente no draw.io, também conhecido como diagrams.net.

Devem ser entregues:

- Diagrama Entidade-Relacionamento;
- Diagrama lógico das tabelas;
- Relacionamentos com chaves primárias e estrangeiras;
- Cardinalidades entre as entidades.

Os arquivos deverão ser exportados nos seguintes formatos:

- .drawio, para edição;
- .png ou .pdf, para visualização.

1.4. Criação do Banco de Dados Físico

Cada grupo deverá criar fisicamente o banco de dados usando SQL. O projeto deverá conter comandos como:

```
CREATE DATABASE nome_do_banco ;  
USE nome_do_banco ;  
  
CREATE TABLE tabela (  
    id INT PRIMARY KEY AUTO_INCREMENT ,  
    campo VARCHAR (100) NOT NULL  
);
```

As tabelas deverão conter:

- PRIMARY KEY;
- FOREIGN KEY;
- NOT NULL;
- Tipos de dados adequados;
- Relacionamentos entre tabelas.

1.5. Inserção de Dados

O grupo deverá inserir dados fictícios suficientes para testar o sistema. Exemplo:

```
INSERT INTO aluno ( nome , ra )  
VALUES ( 'Joao Silva ' , '12345 ' );
```

Sugestão mínima de dados:

10 registros nas tabelas principais ;
5 registros nas tabelas intermediarias ;
20 registros em tabelas de movimentacao , como compra , pedido , presença ou

pagamento .

3

1.6. Consultas SQL

O projeto deverá conter muitas consultas SQL.

Sugestão:

- 10 consultas simples;
- 10 consultas com duas tabelas;
- 10 consultas com três ou mais tabelas;
- 5 consultas com filtros usando WHERE;
- 5 consultas com ORDER BY;
- 5 consultas com GROUP BY.

As consultas deverão respeitar a seguinte regra:

- Não usar JOIN;
- Não usar SUBSELECT.

1.7. Modelo de Consulta Permitido

```
SELECT tabela1 . campo , tabela2 . campo , tabela3 . campo  
FROM tabela1 , tabela2 , tabela3  
WHERE tabela1 . id = tabela2 . id_tabela1  
AND tabela2 . id = tabela3 . id_tabela2 ;
```

1.8. Modelo de Consulta Proibido com JOIN

```
SELECT aluno . nome , turma . nome  
FROM aluno  
INNER JOIN turma ON aluno . id_turma = turma . id_turma ;
```

1.9. Modelo de Consulta Proibido com SUBSELECT

```
SELECT nome
FROM aluno
WHERE id_turma IN (
    SELECT id_turma
    FROM turma
);
```

4

2. Projeto 1: Sistema de Cantina Escolar com Créditos Pré-Pagos

2.1. Ideia Geral

Sistema para controlar alunos, responsáveis, créditos, produtos da cantina e compras realizadas.

O responsável adiciona saldo ao aluno, e o aluno utiliza esse saldo para comprar produtos na cantina escolar.

2.2. Tabelas Sugeridas

```
responsavel
aluno
turma
produto
categoria_produto
recarga
compra
item_compra
funcionario
forma_pagamento
```

2.3. Relacionamentos Principais

- Um responsável pode ter vários alunos;
- Um aluno pertence a uma turma;
- Um aluno pode receber várias recargas;
- Um aluno pode realizar várias compras;
- Uma compra pode ter vários itens;

- Cada item de compra está ligado a um produto;
- Um funcionário registra a compra;
- Um produto pertence a uma categoria;
- Uma recarga possui uma forma de pagamento.

5

2.4. Diagramas Obrigatórios no draw.io

O grupo deverá criar:

- Diagrama Entidade-Relacionamento;
- Diagrama lógico com as tabelas;
- Cardinalidade entre responsável e aluno;
- Cardinalidade entre aluno e compra;
- Cardinalidade entre compra e item_compra;
- Cardinalidade entre produto e item_compra;
- Cardinalidade entre categoria_produto e produto.

2.5. Banco de Dados Físico

```
CREATE DATABASE cantina_escolar ;
USE cantina_escolar ;
```

2.6. Exemplos de Consultas sem JOIN e sem SUBSELECT

Listar alunos com seus responsáveis:

```
SELECT aluno . nome , responsavel . nome
FROM aluno , responsavel
WHERE aluno . id_responsavel = responsavel . id_responsavel ;
```

Listar compras com nome do aluno e funcionário:

```
SELECT compra . id_compra , aluno . nome , funcionario . nome , compra .  
    data_compra  
FROM compra , aluno , funcionario  
WHERE compra . id_aluno = aluno . id_aluno  
AND compra . id_funcionario = funcionario . id_funcionario ;
```

Listar produtos comprados por cada aluno:

```
SELECT aluno . nome , produto . nome , item_compra . quantidade ,  
    item_compra . valor_unitario  
FROM aluno , compra , item_compra , produto  
WHERE aluno . id_aluno = compra . id_aluno  
AND compra . id_compra = item_compra . id_compra  
AND item_compra . id_produto = produto . id_produto ;
```

6

2.7. Justificativa

É um projeto moderno, com lógica financeira simples e várias tabelas relacionadas. Permite trabalhar com saldo, compras, produtos, responsáveis, alunos e histórico.

3. Projeto 2: Sistema de Gestão de Presença Escolar

3.1. Ideia Geral

Sistema para controlar alunos, turmas, professores, disciplinas, aulas e presença dos alunos.

3.2. Tabelas Sugeridas

aluno
turma
professor
disciplina
aula
presenca
curso
periodo
sala
matricula

3.3. Relacionamentos Principais

- Um curso possui várias turmas;
- Uma turma possui vários alunos;
- Um aluno possui matrícula;
- Um professor ministra disciplinas;
- Uma disciplina possui várias aulas;
- Uma aula pertence a uma turma;
- Uma aula ocorre em uma sala;
- A presença relaciona aluno e aula.

7

3.4. Diagramas Obrigatórios no draw.io

O grupo deverá criar:

- Diagrama Entidade-Relacionamento;
- Diagrama lógico das tabelas;
- Relacionamento entre curso e turma;
- Relacionamento entre turma e aluno;
- Relacionamento entre professor, disciplina e aula;
- Relacionamento entre aula, aluno e presença;
- Cardinalidades entre as entidades.

3.5. Banco de Dados Físico

```
CREATE DATABASE presença_escolar ;  
USE presença_escolar ;
```

3.6. Exemplos de Consultas sem JOIN e sem SUBSELECT

Listar alunos de uma turma:

```
SELECT aluno . nome , turma . nome  
FROM aluno , turma  
WHERE aluno . id_turma = turma . id_turma ;
```

Listar presença dos alunos em determinada aula:

```
SELECT aluno . nome , aula . data_aula , presenca . status  
FROM aluno , aula , presenca  
WHERE aluno . id_aluno = presenca . id_aluno  
AND aula . id_aula = presenca . id_aula ;
```

Listar professor, disciplina e turma:

```
SELECT professor . nome , disciplina . nome , turma . nome  
FROM professor , disciplina , turma , aula  
WHERE professor . id_professor = aula . id_professor  
AND disciplina . id_disciplina = aula . id_disciplina  
AND turma . id_turma = aula . id_turma ;
```

3.7. Justificativa

É excelente para modelagem educacional e permite vários relatórios sobre presença, faltas, disciplinas, professores e turmas.

4. Projeto 3: Sistema de Clínica Médica

4.1. Ideia Geral

Sistema para controlar pacientes, médicos, especialidades, consultas, exames, receitas e pagamentos.

4.2. Tabelas Sugeridas

paciente
medico
especialidade
consulta
exame
pedido_exame
receita
medicamento
item_receita
pagamento
convenio

4.3. Relacionamentos Principais

- Um médico pertence a uma especialidade;
- Um paciente pode ter várias consultas;
- Um médico atende várias consultas;
- Uma consulta pode gerar receita;
- Uma receita possui vários medicamentos;
- Uma consulta pode gerar pedidos de exame;
- Um paciente pode possuir convênio;
- Uma consulta pode gerar pagamento.

9

4.4. Diagramas Obrigatórios no draw.io

O grupo deverá criar:

- Diagrama Entidade-Relacionamento;
- Diagrama lógico das tabelas;
- Relacionamento entre paciente e consulta;
- Relacionamento entre médico e especialidade;
- Relacionamento entre consulta e receita;
- Relacionamento entre receita e medicamento;

- Relacionamento entre consulta e exame;
- Relacionamento entre consulta e pagamento.

4.5. Banco de Dados Físico

```
CREATE DATABASE clinica_medica ;
USE clinica_medica ;
```

4.6. Exemplos de Consultas sem JOIN e sem SUBSELECT

Listar consultas com paciente e médico:

```
SELECT paciente . nome , medico . nome , consulta . data_consulta FROM
paciente , medico , consulta
WHERE paciente . id_paciente = consulta . id_paciente
AND medico . id_medico = consulta . id_medico ;
```

Listar medicamentos prescritos para pacientes:

```
SELECT paciente . nome , medicamento . nome , item_receita . dosagem FROM
paciente , consulta , receita , item_receita , medicamento WHERE paciente .
id_paciente = consulta . id_paciente
AND consulta . id_consulta = receita . id_consulta
AND receita . id_receita = item_receita . id_receita
AND item_receita . id_medicamento = medicamento . id_medicamento ;
```

Listar exames solicitados por médico:

10

```
SELECT medico . nome , paciente . nome , exame . nome , pedido_exame .
data_pedido
FROM medico , paciente , consulta , pedido_exame , exame WHERE
medico . id_medico = consulta . id_medico
AND paciente . id_paciente = consulta . id_paciente
AND consulta . id_consulta = pedido_exame . id_consulta
AND pedido_exame . id_exame = exame . id_exame ;
```

4.7. Justificativa

É um projeto bastante completo, com tabelas relacionadas e regras reais de atendimento médico, receitas, exames e pagamentos.

5. Projeto 4: Sistema de Biblioteca Universitária

5.1. Ideia Geral

Sistema para controlar livros, autores, editoras, alunos, professores, empréstimos, reservas e multas.

5.2. Tabelas Sugeridas

usuario
tipo_usuario
livro
autor
livro_autor
editora
categoria
exemplar
emprestimo
reserva
multa

5.3. Relacionamentos Principais

- Um livro pode ter vários autores;
- Um autor pode escrever vários livros;
- Um livro possui vários exemplares;
- Um usuário pode fazer vários empréstimos;
- Um empréstimo envolve um exemplar;
- Um usuário pode fazer reservas;
- Um atraso pode gerar multa;
- Um livro pertence a uma categoria;

- Um livro pertence a uma editora.

5.4. Diagramas Obrigatórios no draw.io

O grupo deverá criar:

- Diagrama Entidade-Relacionamento;
- Diagrama lógico das tabelas;
- Relacionamento muitos-para-muitos entre livro e autor;
- Relacionamento entre livro e exemplar;
- Relacionamento entre usuário e empréstimo;
- Relacionamento entre empréstimo e multa;
- Relacionamento entre usuário e reserva.

5.5. Banco de Dados Físico

```
CREATE DATABASE biblioteca_universitaria ;  
USE biblioteca_universitaria ;
```

5.6. Exemplos de Consultas sem JOIN e sem SUBSELECT

Listar livros com autores:

```
SELECT livro . titulo , autor . nome  
FROM livro , autor , livro_autor  
WHERE livro . id_livro = livro_autor . id_livro  
AND autor . id_autor = livro_autor . id_autor ;
```

Listar empréstimos com usuário e livro:

12

```
SELECT usuario . nome , livro . titulo , emprestimo . data_emprestimo FROM  
usuario , livro , exemplar , emprestimo  
WHERE usuario . id_usuario = emprestimo . id_usuario  
AND exemplar . id_exemplar = emprestimo . id_exemplar  
AND livro . id_livro = exemplar . id_livro ;
```

Listar multas de usuários:

```
SELECT usuario . nome , multa . valor , multa . status
FROM usuario , emprestimo , multa
WHERE usuario . id_usuario = emprestimo . id_usuario
AND emprestimo . id_emprestimo = multa . id_emprestimo ;
```

5.7. Justificativa

É um projeto clássico, muito bom para trabalhar relacionamento muitos-para-muitos e regras de empréstimo, reserva e multa.

6. Projeto 5: Sistema de Hotelaria

6.1. Ideia Geral

Sistema para controlar hóspedes, quartos, reservas, pagamentos, serviços, consumo e funcionários.

6.2. Tabelas Sugeridas

```
hospede
quarto
tipo_quarto
reserva
checkin
checkout
pagamento
servico
consumo_servico
funcionario
forma_pagamento
```

13

6.3. Relacionamentos Principais

- Um hóspede pode fazer várias reservas;
- Um quarto pertence a um tipo de quarto;
- Uma reserva está ligada a um quarto;
- Uma reserva pode gerar check-in e check-out;

- Uma reserva pode ter pagamentos;
- Um hóspede pode consumir serviços;
- Um funcionário pode registrar serviços e atendimentos.

6.4. Diagramas Obrigatórios no draw.io

O grupo deverá criar:

- Diagrama Entidade-Relacionamento;
- Diagrama lógico das tabelas;
- Relacionamento entre hóspede e reserva;
- Relacionamento entre reserva e quarto;
- Relacionamento entre reserva, check-in e check-out;
- Relacionamento entre reserva e pagamento;
- Relacionamento entre reserva e consumo de serviços.

6.5. Banco de Dados Físico

```
CREATE DATABASE hotelaria ;
USE hotelaria ;
```

6.6. Exemplos de Consultas sem JOIN e sem SUBSELECT

Listar reservas com hóspedes e quartos:

```
SELECT hospede . nome , quarto . numero , reserva . data_inicio , reserva . data_fim
FROM hospede , quarto , reserva
WHERE hospede . id_hospede = reserva . id_hospede
AND quarto . id_quarto = reserva . id_quarto ;
```

14

Listar consumo de serviços por hóspede:

```
SELECT hospede . nome , servico . nome , consumo_servico . quantidade FROM
hospede , reserva , consumo_servico , servico
WHERE hospede . id_hospede = reserva . id_hospede
AND reserva . id_reserva = consumo_servico . id_reserva
```

```
AND servico . id_servico = consumo_servico . id_servico ;
```

Listar pagamentos de reservas:

```
SELECT hospede . nome , reserva . id_reserva , pagamento . valor FROM  
hospede , reserva , pagamento  
WHERE hospede . id_hospede = reserva . id_hospede  
AND reserva . id_reserva = pagamento . id_reserva ;
```

6.7. Justificativa

É uma boa opção por envolver reservas, quartos, hóspedes, pagamentos, serviços e funcionários.

7. Projeto 6: Sistema de Loja Virtual

7.1. Ideia Geral

Sistema para controlar clientes, produtos, categorias, pedidos, pagamentos, entregas e avaliações.

7.2. Tabelas Sugeridas

```
cliente  
endereco  
produto  
categoria  
pedido  
item_pedido  
pagamento  
forma_pagamento  
entrega  
status_pedido  
avaliacao
```

7.3. Relacionamentos Principais

- Um cliente pode ter vários endereços;
- Um produto pertence a uma categoria;

- Um cliente pode fazer vários pedidos;
- Um pedido possui vários itens;
- Cada item está ligado a um produto;
- Um pedido possui pagamento;
- Um pedido possui entrega;
- Um produto pode receber avaliações.

7.4. Diagramas Obrigatórios no draw.io

O grupo deverá criar:

- Diagrama Entidade-Relacionamento;
- Diagrama lógico das tabelas;
- Relacionamento entre cliente e pedido;
- Relacionamento entre pedido e item_pedido;
- Relacionamento entre produto e item_pedido;
- Relacionamento entre pedido e pagamento;
- Relacionamento entre pedido e entrega;
- Relacionamento entre produto e avaliação.

7.5. Banco de Dados Físico

```
CREATE DATABASE loja_virtual ;
USE loja_virtual ;
```

16

7.6. Exemplos de Consultas sem JOIN e sem SUBSELECT

Listar pedidos com cliente e status:

```
SELECT cliente . nome , pedido . id_pedido , status_pedido . descricao FROM
cliente , pedido , status_pedido
WHERE cliente . id_cliente = pedido . id_cliente
AND pedido . id_status = status_pedido . id_status ;
```

Listar produtos de cada pedido:

```
SELECT cliente . nome , pedido . id_pedido , produto . nome , item_pedido . quantidade
FROM cliente , pedido , item_pedido , produto
WHERE cliente . id_cliente = pedido . id_cliente
AND pedido . id_pedido = item_pedido . id_pedido
AND produto . id_produto = item_pedido . id_produto ;
```

Listar pagamentos dos pedidos:

```
SELECT cliente . nome , pedido . id_pedido , pagamento . valor ,
       forma_pagamento . descricao
FROM cliente , pedido , pagamento , forma_pagamento
WHERE cliente . id_cliente = pedido . id_cliente
AND pedido . id_pedido = pagamento . id_pedido
AND pagamento . id_forma_pagamento = forma_pagamento .
    id_forma_pagamento ;
```

7.7. Justificativa

É um projeto muito completo, pois trabalha com lógica parecida com e-commerce

real. 8. Projeto 7: Sistema de Oficina Mecânica

8.1. Ideia Geral

Sistema para controlar clientes, veículos, mecânicos, ordens de serviço, peças, serviços realizados e pagamentos.

8.2. Tabelas Sugeridas

cliente
veiculo
marca

modelo
mecanico
ordem_servico
servico

item_servico
peca
item_peca
pagamento

8.3. Relacionamentos Principais

- Um cliente pode ter vários veículos;
- Um veículo pertence a uma marca e modelo; • Uma ordem de serviço pertence a um veículo; • Um mecânico executa serviços;
- Uma ordem de serviço pode ter vários serviços; • Uma ordem pode usar várias peças;
- Uma ordem gera pagamento.

8.4. Diagramas Obrigatórios no draw.io O

grupo deverá criar:

- Diagrama Entidade-Relacionamento;
- Diagrama lógico das tabelas;
- Relacionamento entre cliente e veículo;
- Relacionamento entre veículo, marca e modelo; • Relacionamento entre veículo e ordem de serviço; • Relacionamento entre ordem de serviço e serviços; • Relacionamento entre ordem de serviço e peças; • Relacionamento entre ordem de serviço e pagamento.

8.5. Banco de Dados Físico

```
CREATE DATABASE oficina_mecanica ;  
USE oficina_mecanica ;
```

8.6. Exemplos de Consultas sem JOIN e sem SUBSELECT

Listar veículos por cliente:

```
SELECT cliente . nome , veiculo . placa , marca . nome , modelo . nome FROM  
cliente , veiculo , marca , modelo  
WHERE cliente . id_cliente = veiculo . id_cliente  
AND veiculo . id_marca = marca . id_marca  
AND veiculo . id_modelo = modelo . id_modelo ;
```

Listar serviços realizados em uma ordem:

```
SELECT ordem_servico . id_ordem , servico . nome , mecanico . nome FROM  
ordem_servico , item_servico , servico , mecanico  
WHERE ordem_servico . id_ordem = item_servico . id_ordem AND  
item_servico . id_servico = servico . id_servico  
AND item_servico . id_mecanico = mecanico . id_mecanico ;
```

Listar peças usadas por veículo:

```
SELECT cliente . nome , veiculo . placa , peca . nome , item_peca . quantidade FROM  
cliente , veiculo , ordem_servico , item_peca , peca WHERE cliente . id_cliente = veiculo .  
id_cliente  
AND veiculo . id_veiculo = ordem_servico . id_veiculo  
AND ordem_servico . id_ordem = item_peca . id_ordem  
AND item_peca . id_peca = peca . id_peca ;
```

8.7. Justificativa

É excelente para relacionar cliente, veículo, mecânico, serviço, peça e

pagamento.

9. Projeto 8: Sistema de Academia

9.1. Ideia Geral

Sistema para controlar alunos, planos, professores, treinos, exercícios, avaliações físicas e pagamentos.

9.2. Tabelas Sugeridas

aluno
plano
matricula
professor
treino
exercicio
treino_exercicio
avaliacao_fisica
pagamento
modalidade
aula_coletiva
presenca_aula

9.3. Relacionamentos Principais

- Um aluno possui matrícula;
- Uma matrícula está ligada a um plano;
- Um professor monta treinos;
- Um treino possui vários exercícios;
- Um exercício pode estar em vários treinos;
- Um aluno possui avaliações físicas;
- Um aluno realiza pagamentos;
- Uma modalidade pode ter aulas coletivas;
- Alunos podem marcar presença em aulas coletivas.

9.4. Diagramas Obrigatórios no draw.io

O grupo deverá criar:

- Diagrama Entidade-Relacionamento;
- Diagrama lógico das tabelas;
- Relacionamento entre aluno, matrícula e plano;

- Relacionamento entre aluno, treino e professor;

20

- Relacionamento muitos-para-muitos entre treino e exercício;
- Relacionamento entre aluno e avaliação física;
- Relacionamento entre modalidade, aula coletiva e presença.

9.5. Banco de Dados Físico

```
CREATE DATABASE academia ;
USE academia ;
```

9.6. Exemplos de Consultas sem JOIN e sem SUBSELECT

Listar alunos e seus planos:

```
SELECT aluno . nome , plano . nome , matricula . data_inicio FROM
aluno , plano , matricula
WHERE aluno . id_aluno = matricula . id_aluno
AND plano . id_plano = matricula . id_plano ;
```

Listar exercícios de cada treino:

```
SELECT aluno . nome , treino . nome , exercicio . nome , treino_exercicio . series ,
treino_exercicio . repeticoes
FROM aluno , treino , treino_exercicio , exercicio
WHERE aluno . id_aluno = treino . id_aluno
AND treino . id_treino = treino_exercicio . id_treino
AND exercicio . id_exercicio = treino_exercicio . id_exercicio ;
```

Listar pagamentos dos alunos:

```
SELECT aluno . nome , pagamento . valor , pagamento . data_pagamento
FROM aluno , matricula , pagamento
WHERE aluno . id_aluno = matricula . id_aluno
AND matricula . id_matricula = pagamento . id_matricula ;
```

9.7. Justificativa

É ótimo para demonstrar relacionamento muitos-para-muitos entre treino e exercício,

além de planos, pagamentos e avaliações.

21

10. Projeto 9: Sistema de Delivery de Restaurante

10.1. Ideia Geral

Sistema para controlar clientes, restaurantes, produtos, pedidos, entregadores, pagamentos e avaliações.

10.2. Tabelas Sugeridas

cliente
endereco
restaurante
categoria_restaurante
produto
categoria_produto
pedido
item_pedido
pagamento
entregador
entrega
avaliacao

10.3. Relacionamentos Principais

- Um cliente pode ter vários endereços;
- Um restaurante possui vários produtos;
- Um restaurante pertence a uma categoria;
- Um pedido pertence a um cliente;
- Um pedido possui vários itens;
- Cada item se refere a um produto;
- Um pedido possui uma entrega;

- Uma entrega é feita por um entregador;
- O cliente pode avaliar restaurante e entrega.

22

10.4. Diagramas Obrigatórios no draw.io

O grupo deverá criar:

- Diagrama Entidade-Relacionamento;
- Diagrama lógico das tabelas;
- Relacionamento entre cliente e endereço;
- Relacionamento entre restaurante e produto;
- Relacionamento entre pedido e item_pedido;
- Relacionamento entre pedido e entrega;
- Relacionamento entre entrega e entregador;
- Relacionamento entre cliente, restaurante e avaliação.

10.5. Banco de Dados Físico

```
CREATE DATABASE delivery_restaurante ;
USE delivery_restaurante ;
```

10.6. Exemplos de Consultas sem JOIN e sem SUBSELECT

Listar pedidos com cliente e restaurante:

```
SELECT cliente . nome , restaurante . nome , pedido . data_pedido FROM
cliente , restaurante , pedido
WHERE cliente . id_cliente = pedido . id_cliente
AND restaurante . id_restaurante = pedido . id_restaurante ;
```

Listar itens dos pedidos:

```
SELECT cliente . nome , produto . nome , item_pedido . quantidade FROM
cliente , pedido , item_pedido , produto
WHERE cliente . id_cliente = pedido . id_cliente
AND pedido . id_pedido = item_pedido . id_pedido
AND item_pedido . id_produto = produto . id_produto ;
```

Listar entregas com entregador:

23

```
SELECT cliente . nome , entregador . nome , entrega . status FROM
cliente , pedido , entrega , entregador
WHERE cliente . id_cliente = pedido . id_cliente
AND pedido . id_pedido = entrega . id_pedido
AND entrega . id_entregador = entregador . id_entregador ;
```

10.7. Justificativa

É um projeto muito próximo de sistemas reais de delivery e permite muitos relacionamentos.

11. Projeto 10: Sistema de Eventos e Inscrições

11.1. Ideia Geral

Sistema para controlar eventos, participantes, palestrantes, salas, inscrições, certificados e pagamentos.

11.2. Tabelas Sugeridas

participante
evento
categoria_evento
palestrante
palestra
sala
inscricao
pagamento
certificado
presenca
organizador

11.3. Relacionamentos Principais

- Um evento possui várias palestras;
- Uma palestra possui um palestrante;
- Uma palestra ocorre em uma sala;
- Um participante pode se inscrever em vários eventos;

24

- Um evento pode ter vários participantes;
- Uma inscrição pode gerar pagamento;
- A presença pode gerar certificado;
- Um organizador administra eventos.

11.4. Diagramas Obrigatórios no draw.io

O grupo deverá criar:

- Diagrama Entidade-Relacionamento;
- Diagrama lógico das tabelas;
- Relacionamento entre evento, palestra e palestrante;
- Relacionamento entre palestra e sala;
- Relacionamento entre participante, evento e inscrição;
- Relacionamento entre inscrição e pagamento;

- Relacionamento entre inscrição, presença e certificado;
- Relacionamento entre organizador e evento.

11.5. Banco de Dados Físico

```
CREATE DATABASE eventos_inscricoes ;
USE eventos_inscricoes ;
```

11.6. Exemplos de Consultas sem JOIN e sem SUBSELECT

Listar participantes inscritos em eventos:

```
SELECT participante . nome , evento . nome , inscricao . data_inscricao FROM
participante , evento , inscricao
WHERE participante . id_participante = inscricao . id_participante AND evento .
id_evento = inscricao . id_evento ;
```

Listar palestras com palestrantes e salas:

25

```
SELECT evento . nome , palestra . titulo , palestrante . nome , sala . nome FROM
evento , palestra , palestrante , sala
WHERE evento . id_evento = palestra . id_evento
AND palestra . id_palestrante = palestrante . id_palestrante AND palestra .
id_sala = sala . id_sala ;
```

Listar certificados emitidos:

```
SELECT participante . nome , evento . nome , certificado . codigo FROM
participante , evento , inscricao , certificado
WHERE participante . id_participante = inscricao . id_participante AND evento .
id_evento = inscricao . id_evento
AND inscricao . id_inscricao = certificado . id_inscricao ;
```

11.7. Justificativa

É um projeto completo para modelagem, pois envolve inscrições, pagamentos,

presenças, palestras, certificados e eventos.

26

12. Modelo Obrigatório de Entrega

Cada grupo deverá entregar os seguintes itens:

12.1. Documento do Projeto

O documento deverá conter:

- Nome do sistema;
- Objetivo;
- Descrição do problema;
- Regras de negócio;
- Lista de entidades;

- Lista de atributos;
- Relacionamentos;
- Cardinalidades.

12.2. Diagramas no draw.io

Os diagramas deverão ser obrigatoriamente feitos no [draw.io/diagrams.net](https://draw.io). Devem ser entregues:

- Arquivo editável .drawio;
- Imagem .png ou arquivo .pdf;
- Diagrama conceitual;
- Diagrama lógico;
- Relacionamentos com chaves primárias e estrangeiras.

12.3. Modelo Físico do Banco

O grupo deverá entregar um arquivo .sql contendo:

- CREATE DATABASE;
- USE;

27

- CREATE TABLE;
- Chaves primárias;
- Chaves estrangeiras;
- Restrições;
- Comandos INSERT.

12.4. Massa de Dados

O banco deverá conter dados suficientes para teste.

Sugestão mínima:

10 registros nas tabelas principais ;
5 registros nas tabelas intermediarias ;
20 registros em tabelas de movimentacao .

12.5. Consultas SQL

Cada grupo deverá criar várias consultas.

Sugestão:

10 consultas simples ;
10 consultas com duas tabelas ;
10 consultas com tres ou mais tabelas ;
5 consultas com filtros usando WHERE ;
5 consultas com ORDER BY ;
5 consultas com GROUP BY .

Todas as consultas deverão seguir a regra:

- Não usar JOIN;
- Não usar SUBSELECT.

13. Melhores Escolhas de Projeto

13.1. Projeto Mais Completo

Sistema de Eventos e Inscrições

Esse projeto é muito completo, pois envolve eventos, palestras, participantes, inscrições, pagamentos, presenças e certificados.

28

13.2. Projeto Mais Fácil de Entender

Sistema de Cantina Escolar com Créditos Pré-Pagos

Esse projeto possui uma lógica clara: o aluno recebe crédito, compra produto e o saldo é descontado.

13.3. Projeto Mais Clássico para Banco de Dados

Sistema de Biblioteca Universitária

Esse projeto trabalha com autores, livros, exemplares, empréstimos, reservas e multas.

13.4. Projeto Melhor para Área Escolar

Sistema de Gestão de Presença Escolar

Esse projeto envolve aluno, turma, disciplina, professor, aula e presença.

13.5. Projeto com Mais Cara de Sistema Comercial

Sistema de Loja Virtual

Esse projeto envolve cliente, produto, pedido, pagamento, entrega e avaliação.

14. Conclusão

Os projetos apresentados permitem aplicar os principais conceitos de banco de dados, incluindo modelagem conceitual, modelagem lógica, criação do banco físico, relacionamentos entre tabelas, chaves primárias, chaves estrangeiras e consultas SQL.

A obrigatoriedade do uso do draw.io garante que os grupos desenvolvam diagramas claros e organizados. A criação do banco físico em SQL permite que o projeto saia da modelagem teórica e seja testado de forma prática.

Além disso, a restrição de não utilizar JOIN nem SUBSELECT nas consultas estimula o estudante a compreender melhor os relacionamentos entre as tabelas utilizando condições no WHERE.